PYTHON INNOVATION HACKATHON / SMART COMMUNITY

Kanban e Checkpoint. CivicLink.

Come è cambiato il lavoro dal piano iniziale al risultato finale: board Kanban, tre checkpoint, riflessione e contributi individuali.

EVENTO

Python Innovation Hackathon

TEMA

Smart Community

DATA

09/11/2026

GRUPPO

Davide Storelli · Pietro Russo · Simone Stellabotte · Emanuele Timbo

Stato iniziale. Pianificato.

Le attività così come erano state pianificate all'inizio del progetto.

PIANIFICATE	CHECKPOINT CP1	CHECKPOINT CP2	CHECKPOINT CP3
18	8	14	18
attività totali	completate · 44%	completate · 78%	completate · 100%

TO DO (18)	IN PROGRESS	DONE
Creare struttura cartelle (data/, src/, frontend/)	*Nessuna attività*	*Nessuna attività*
Scrivere modulo segnalazioni.py		
Scrivere modulo oggetti.py		
Scrivere modulo statistiche.py		
Scrivere modulo gestione_file.py		
Creare menu interattivo in main.py con opzioni 1-9		
Implementare validazione input (campi obbligatori)		
Implementare validazione categorie e priorità		
Gestire salvataggio dati in JSON		
Implementare export CSV per segnalazioni		
Implementare export CSV per oggetti		
Costruire dashboard HTML base		
Aggiungere CSS per stile		
Implementare JavaScript con FileReader		
Testare inserimento segnalazioni		
Testare inserimento oggetti		
Testare visualizzazione dashboard		
Scrivere documentazione README		

Stato finale. Completato.

Lo stato finale di tutte le attività, con quelle effettivamente realizzate.

COMPLETATE	MAI INIZIATE	BLOCCATE
23	0	0
5 in più rispetto al piano iniziale	nessuna attività saltata	nessun blocco aperto

TO DO	IN PROGRESS	DONE 23
Nessuna attività	*Nessuna attività*	✓ Struttura cartelle creata (data/, data/exports/, src/, frontend/)
		✓ Modulo segnalazioni.py con CRUD completo
		✓ Modulo oggetti.py con CRUD completo
		✓ Modulo statistiche.py per calcolo indicatori
		✓ Modulo gestione_file.py con backup automatico .bak
		✓ Menu interattivo con 10 opzioni funzionanti
		✓ Validazione con _input_valido() e _input_non_vuoto()
		✓ Salvataggio JSON con backup automatico
		✓ Export CSV per segnalazioni
		✓ Export CSV per oggetti
		✓ Dashboard HTML professionale
		✓ CSS con design responsive e moderno
		✓ JavaScript con FileReader API
		✓ Grafici a barre per categorie segnalazioni
		✓ Grafici a ciambella per distribuzione stati
		✓ Filtri interattivi (categoria, stato, priorità)
		✓ Toast notifications per feedback utente
		✓ Indicatore stato dati nell'header
		✓ 17 test automatici con unittest (100% passati)
		✓ Test manuali completi
		✓ Documentazione: README, Project Brief, Kanban, Report Test
		✓ Bug fix: campi vuoti, ID inesistenti, JSON corrotto
		✓ Conferma prima di sovrascrivere dati demo

CP1. Fine prima giornata.

Obiettivo: Setup progetto e backend core funzionante

COSA AVEVAMO PIANIFICATO

- **Struttura cartelle completata**
- **Moduli Python funzionanti (almeno segnalazioni e oggetti)**
- **Menu base con inserimento e visualizzazione**

COSA ABBIAMO EFFETTIVAMENTE FATTO

- ✓ **Struttura cartelle creata correttamente**
- ✓ **Tutti e 4 i moduli Python implementati (segnalazioni, oggetti, statistiche, gestione_file)**
- ✓ **Menu completo con 10 opzioni**
- ✓ **Validazione input funzionante**

PROBLEMI INCONTRATI E DECISIONI

PROBLEMA 1

Inizialmente avevamo pensato di usare solo CSV, ma ci siamo resi conto che per la dashboard serviva una struttura più ricca (statistiche + liste annidate).

DECISIONE

Abbiamo scelto JSON come formato primario e CSV come secondario. Questo ci ha permesso di esportare un unico file dashboard_data.json con tutti i dati necessari al frontend.

PROBLEMA 2

I percorsi relativi ai file JSON non funzionavano se il programma veniva lanciato da una directory diversa.

DECISIONE

Abbiamo usato Path(__file__).resolve().parent.parent per calcolare i percorsi assoluti in modo affidabile.

CONFRONTO PIANO VS RISULTATO

ASPETTO	PIANO	RISULTATO	SCOSTAMENTO
Moduli Python	2 (segnalazioni, oggetti)	4 (+ statistiche, gestione_file)	✓ Migliore del previsto
Validazione	Base	Completa con loop di re-input	✓ Migliore del previsto
Backup dati	Non previsto	Backup .bak automatico	✓ Aggiunta utile
Tempo stimato	4 ore	3 ore	✓ Più veloce del previsto

CP2. Metà seconda giornata.

Obiettivo: Frontend dashboard e integrazione con backend

COSA AVEVAMO PIANIFICATO

- **Dashboard HTML base con tabelle**
- **Caricamento JSON funzionante**
- **Grafici semplici (opzionali)**

COSA ABBIAMO EFFETTIVAMENTE FATTO

- ✓ **Dashboard professionale con sidebar, metriche, grafici a barre e donut**
- ✓ **Caricamento JSON tramite FileReader API**
- ✓ **Filtri interattivi per categoria, stato, priorità**
- ✓ **Toast notifications per feedback utente**
- ✓ **Design responsive (desktop, tablet, mobile)**

PROBLEMI INCONTRATI E DECISIONI

PROBLEMA 1

Il primo caricamento del JSON nella dashboard falliva perché le chiavi avevano spazi finali (es. "meta " invece di "meta").

DECISIONE

Abbiamo identificato che il problema era nel file JSON generato manualmente. Abbiamo fornito un JSON corretto e aggiunto controlli più robusti nel frontend.

PROBLEMA 2

I grafici a ciambella non mostravano il testo centrato correttamente.

DECISIONE

Abbiamo rifatto il CSS usando flexbox invece di position absolute, ottenendo una centratura perfetta.

PROBLEMA 3

La dashboard non dava feedback visivo quando i dati venivano caricati.

DECISIONE

Abbiamo aggiunto toast notifications (verde per successo, rosso per errore) e un indicatore di stato nell'header ("Dati caricati" / "In attesa").

CONFRONTO PIANO VS RISULTATO

ASPETTO	PIANO	RISULTATO	SCOSTAMENTO
Dashboard	Base con tabelle	Professionale con grafici e filtri	✓ Molto migliore
Grafici	Opzionali	Implementati (barre + donut)	✓ Aggiunti
Responsive	Non previsto	Completo	✓ Aggiunto
Feedback utente	Nessuno	Toast + indicatori stato	✓ Aggiunto
Tempo stimato	3 ore	4 ore	! Leggermente più lungo

CP3. Fine progetto.

Obiettivo: Test completi e documentazione finale

COSA AVEVAMO PIANIFICATO

- **Test manuali del flusso principale**
- **Documentazione base (README)**

COSA ABBIAMO EFFETTIVAMENTE FATTO

- ✓ **17 test automatici con unittest (tutti passati)**
- ✓ **Test manuali completi (inserimento, validazione, export, caricamento dashboard)**
- ✓ **Documentazione completa: Project Brief, Kanban, Report Test**
- ✓ **Bug fix documentati (campi vuoti, ID inesistenti, JSON corrotto)**

PROBLEMI INCONTRATI E DECISIONI

PROBLEMA 1

Il test test_menu_oggetti_ripete_input_nome_vuoto falliva perché il codice di menu_visualizza_oggetti() era stato incollato per errore dentro menu_oggetti().

DECISIONE

Abbiamo separato le due funzioni e aggiunto la validazione dell'ID prima di chiedere lo stato.

PROBLEMA 2

La modifica stato con ID inesistente chiedeva comunque il nuovo stato prima di rifiutare.

DECISIONE

Abbiamo aggiunto il controllo di esistenza dell'ID prima di richiedere input aggiuntivi, migliorando l'UX.

PROBLEMA 3

L'opzione 9 (carica demo) sovrascriveva i dati senza conferma.

DECISIONE

Abbiamo aggiunto una richiesta di conferma prima di sovrascrivere i dati esistenti.

CONFRONTO PIANO VS RISULTATO

ASPETTO	PIANO	RISULTATO	SCOSTAMENTO
Test	Manuali	17 automatici + manuali	✓ **Molto migliore**
Documentazione	README	Brief + Kanban + Report Test	✓ **Molto migliore**
Bug fix	Non previsto	4 bug corretti e documentati	✓ **Aggiunto**
Tempo stimato	2 ore	3 ore	! **Leggermente più lungo**

05 / RIFLESSIONE FINALE

Obiettivi raggiunti.

Tutti e 6 gli obiettivi verificabili del Project Brief sono stati raggiunti:

- **Inserimento segnalazione con validazione**
- **Consultazione con filtri (terminale + dashboard)**
- **Aggiornamento stato segnalazione**
- **Registrazione oggetti smarriti/ritrovati**
- **Export JSON e CSV**
- **Dashboard web con caricamento manuale**

PROVE EFFETTUATE

- **17 test automatici** con unittest: tutti passati
- **Test manuali:** inserimento valido, campi vuoti, ID inesistenti, JSON corrotto, caricamento dashboard
- **Test di resilienza:** file JSON errati, input non validi, percorsi diversi

LIMITI RICONOSCIUTI

LIMITE	COSA SIGNIFICA	SCELTA MOTIVATA
Nessuna autenticazione	Il sistema non distingue tra utenti diversi	Non richiesto dalla traccia e avrebbe aggiunto complessità senza valore per la demo
Nessuna notifica automatica	Le segnalazioni non generano email o push notification	Richiederebbe server e API esterne, violando il vincolo "100% offline"
Nessuna mappa interattiva	Le segnalazioni non sono geolocalizzate su una mappa	Richiederebbe librerie esterne (Leaflet, Google Maps)

MIGLIORAMENTI FUTURI

- Notifiche email per segnalazioni ad alta priorità
- Mappa interattiva con clustering delle segnalazioni per zona
- Sistema di priorità automatica basato sulla frequenza delle segnalazioni simili
- Export PDF dei report per la stampa

06 / CONTRIBUTI INDIVIDUALI

Chi ha fatto cosa.

MEMBRO	RESPONSABILITÀ	CONTRIBUTI PRINCIPALI
Davide Storelli e Pietro Russo	Frontend + Documentazione	Dashboard HTML/CSS/JS, Project Brief, Kanban, Report Test
Simone Stellabotte	Backend Python	Moduli segnalazioni.py, oggetti.py, main.py, validazione input
Emanuele Timbo	Gestione dati + Test	gestione_file.py, statistiche.py, unittest, test manuali, bug fix

COME ABBIAMO LAVORATO

01

Prima giornata

Setup struttura, backend core, menu interattivo

02

Seconda giornata (mattina)

Frontend dashboard, integrazione JSON

03

Seconda giornata (pomeriggio)

Test, bug fix, documentazione

DECISIONI DI GRUPPO

- Scelta JSON come formato primario (discussione: CSV vs JSON)
- Aggiunta backup automatico .bak (proposta di Emanuele Timbo)
- Design responsive (proposta di Davide Storelli e Pietro Russo)
- Test automatici con unittest (proposta di Emanuele Timbo)

CHECKLIST DI CONSEGNA

- ✓ **Kanban iniziale e finale**
- ✓ **Tre checkpoint (CP1, CP2, CP3) con confronto concreto**
- ✓ **Decisioni motivate documentate**
- ✓ **Riflessione finale con obiettivi, prove, limiti**
- ✓ **Contributi individuali dichiarati**
- ✓ **Problemi e soluzioni documentati**